

Lab 08 · La inspección antes de abrir

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

90 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que un test que nunca se ha puesto rojo no demuestra nada — se rompe la producción a propósito para verlo—, y que probar con Spring cuesta: **0,089 s los cuatro casos sin Spring, 1,26 s uno solo con el contexto entero.**

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. La inspección, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · El primer test, sobre una regla de verdad	3
3.2. Paso 2 · El test que avisa — el momento del laboratorio	5
3.3. Paso 3 · Aislar con un doble	6
3.4. Paso 4 · Los cuatro niveles, y lo que cuesta cada uno	8
4. Lo que aprendiste	10
5. Para profundizar	10
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Todo lo que has comprobado hasta ahora lo has comprobado **mirando la consola**. Eso no se puede repetir, no avisa cuando alguien rompe algo, y no se puede correr mil veces.

Hoy escribes tu primer test. Vas a verlo pasar, **vas a romper el código de producción a propósito para verlo fallar**, y vas a aprender los cuatro niveles a los que se puede probar una aplicación Spring — con el precio de cada uno medido en segundos.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 02 y 03 hechos	Sabes qué es inyección por constructor	Imprescindible en el paso 4
Estar en la carpeta del lab	<code>cd labs/lab-08-testing/practica</code>	El cd no da error
Ninguna base de datos	Este lab no la usa	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en `labs/lab-08-testing/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-08-testing/practica
./mvnw test
```

Este lab no se arranca: se testea. El comando de hoy es `./mvnw test`, y no `spring-boot:run`. No hay puertos que liberar ni procesos que matar.

2 El caso

La oficina de la DGT va a abrir. Antes de dejar entrar al público, **hay que inspeccionarla**.

2.1 La inspección, que es la metáfora de este laboratorio

Se puede inspeccionar a cuatro niveles, y cada uno cuesta distinto.

1. **Una mesa suelta.** Coges la calculadora del mostrador, le das dos números y compruebas el resultado. No hace falta abrir la oficina, ni encender nada, ni que haya nadie. Es un test **unitario**, y tarda milésimas.
2. **La mesa con un proveedor de mentira.** Quieres probar al jefe de compras, pero no quieres depender de lo que el proveedor real tenga hoy en el almacén. Le pones **un proveedor figurante** que entrega exactamente lo que tú digas. Eso es un **doble**, y así los números del test los eliges tú.
3. **La ventanilla, sin abrir la puerta de la calle.** Quieres comprobar que la ventanilla atiende bien: que responde al número correcto, que sella el papel como debe. Simulas a alguien pidiendo por la ventanilla **sin abrir el edificio al público**. Eso es probar la capa web sin levantar el servidor.
4. **Abrir la oficina entera.** Todo montado, todas las salas, el conserje haciendo su ronda. Es la única forma de comprobar que **el edificio se sostiene** — y es la más cara con diferencia.

La regla que sale de aquí: **inspecciona al nivel más barato que responda tu pregunta**. Hoy vas a ver los cuatro y cuánto cuesta cada uno, con el reloj delante.

3 Los pasos

3.1 Paso 1 · El primer test, sobre una regla de verdad

Qué vamos a hacer

Escribir un test de una clase, sin Spring por ninguna parte, sobre el único método del proyecto que tiene una **regla** que alguien puede romper.

Para entenderlo mejor

La mesa suelta: la calculadora de la cotización. Le das un producto y una cantidad, y compruebas el total.

El problema

Comprobar un cálculo arrancando la aplicación y mirando la consola tarda segundos, hay que hacerlo a mano, y **no queda constancia**. Mañana nadie sabe si se comprobó.

Y hay un problema anterior, que es el que decide qué se prueba: **no todo merece un test**. Probar que el catálogo tiene cuatro productos, o que el producto 2 se llama «Tóner negro», no protege nada: son datos de una lista de mentira, y el día que alguien añada un producto ese test se pone rojo sin que nada esté mal.

Lo que sí merece un test es la regla del descuento:

3 o más unidades, 10 %. 10 o más, 20 %.

Ahí hay cuatro cosas que pueden salir mal: los **bordes** (¿3 entra en el 10 %?), el **orden** de los tramos (si se pregunta por ≥ 3 antes que por ≥ 10 , diez unidades se llevan el 10 %), el **redondeo**, y la **entrada inválida**.

La alternativa, y por qué no

- **Un main que imprima el resultado** y mirarlo. Es lo que se hace sin saber que existen los tests, y no falla solo: hay que estar delante.
- **Cuatro @Test copiados y pegados**, uno por caso: el mismo cuerpo cuatro veces, y cuando cambie la regla hay que acordarse de cambiarlo en cuatro sitios.
- **Un @Test con cuatro assertEquals seguidos**: peor que las dos, porque **el primero que falla corta el método** y nunca llegas a saber si los otros tres también estaban mal.
- **Un @ParameterizedTest**, que es lo de aquí: un cuerpo, los datos en una tabla, y JUnit los cuenta como cuatro ejecuciones independientes.

Y una decisión de forma que conviene entender: **esto es Java y JUnit, sin Spring**. Mucha gente cree que testear una aplicación Spring exige levantar Spring. No: la clase que vas a probar es una clase normal con un constructor, y se prueba como cualquier otra.

Se pega

En `practica/src/test/java/cl/dgt/testing/ProductoServiceTest.java`, que llega **declarado y vacío** — fíjate en que va bajo `src/test/java`, no bajo `src/main/java`:

```
    " 1, 5938",      // sin descuento
    " 3, 16033",     // 10 %
    "10, 47504",     // 20 %
    " 0,      0"     // cantidad inválida: lanza
})
void elTotalAplicaElDescuentoPorVolumen(int cantidad, int esperado) {
    if (cantidad <= 0) {
        assertThrows(IllegalArgumentException.class, () ->
            ↪ servicio.totalConDescuento(1L, cantidad));
        return;
    }
}
```

`assertEquals(esperado, obtenido)`: **el primero es lo que esperas**. Al revés, el mensaje de fallo miente.

Y fíjate en que el repositorio es el de verdad, no un doble: `ProductoRepositoryLista` es una lista en memoria, así que no hay nada lento que aislar. El doble llega en el paso 2, cuando tenga algo que demostrar.

Se corre

```
./mvnw test
```

Lo que vas a ver

```
[INFO] Running cl.dgt.testing.ProductoServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s
[INFO] BUILD SUCCESS
```

Un método, cuatro ejecuciones, 0,089 segundos. Guarda ese número: vuelve en el paso 4.

Los números salen de una cuenta que puedes rehacer en un papel: $4990 \times 1,19 = 5938$; por tres, 17.814; menos un 10 %, **16.033**.

Vas bien si...

BUILD SUCCESS y una línea Tests run: 4 con Failures: 0.

Si te atascas

1 · No tests were executed!

El archivo está en src/main/java en vez de src/test/java, o el nombre de la clase no acaba en Test. Maven busca por convención.

2 · cannot find symbol: class ParameterizedTest

Falta import org.junit.jupiter.params.ParameterizedTest;. Ojo al params en medio: es otro paquete que el de @Test.

3 · El test pasa pero no comprueba nada.

Si escribiste el assertEquals con los dos argumentos iguales, siempre pasará. El paso siguiente está justo para descubrir eso.

3.2 Paso 2 · El test que avisa — el momento del laboratorio

Qué vamos a hacer

Romper el código de producción a propósito y mirar qué dice el test.

Para entenderlo mejor

Un detector de humo que nunca ha sonado no está probado: está **sin probar**. La única forma de saber que funciona es echarle humo.

El problema

Un test en verde no demuestra que el código esté bien. Demuestra que **el test pasó**. Y un test mal escrito —que compara algo consigo mismo, o que no llega a ejecutar lo que cree— pasa siempre y no protege de nada.

Un test que nunca se ha puesto rojo podría estar comprobando el aire.

Se hace

Abre `practica/src/main/java/cl/dgt/testing/services/ProductoService.java` y **cambia la tasa del IVA** de 0.19 a 0.10. Es una mentira evidente, y ésa es la gracia.

```
./mvnw test
```

Lo que vas a ver

```
[ERROR] Tests run: 4, Failures: 3, Errors: 0, Skipped: 0 <<< FAILURE!  
[ERROR] elTotalAplicaElDescuentoPorVolumen(int, int)[1] <<< FAILURE!  
[ERROR] elTotalAplicaElDescuentoPorVolumen(int, int)[2] <<< FAILURE!  
[ERROR] elTotalAplicaElDescuentoPorVolumen(int, int)[3] <<< FAILURE!
```

Tres de cuatro. El cuarto —el de cero unidades— sigue verde: no depende del IVA. Eso es lo que gana un test parametrizado sobre cuatro `assertEquals` seguidos.

Y el detalle está en el informe que Surefire deja escrito:

```
cat target/surefire-reports/cl.dgt.testing.ProductoServiceTest.txt
```

```
org.opentest4j.AssertionFailedError: expected: <5938> but was: <5489>
```

```
at
```

```
↪ cl.dgt.testing.ProductoServiceTest.elTotalAplicaElDescuentoPorVolumen(ProductoServiceTest.java
```

Lee el mensaje entero, porque un buen fallo es media reparación:

- **Qué esperaba** (5938) y **qué salió** (5489).
- **Qué test** falló, y **qué caso** de los cuatro.
- **En qué línea.**

Dos líneas, no sesenta, y eso se lo debe este laboratorio a una línea del `pom.xml`: `<trimStackTrace>true</trimStackTrace>`. Sin ella, ese mismo fallo entierra la información útil bajo la pila entera de JUnit y Spring.

Nadie tuvo que arrancar nada ni mirar ninguna consola. **Ahora vuelve a poner 0.19** y corre otra vez: verde. El test está probado.

Vas bien si...

Viste el rojo con `expected: <5938> but was: <5489>` en tres de los cuatro casos, y al deshacer el cambio volvió a verde.

Si te atascas

1 · Cambias el IVA y el test sigue verde.

Éste es el hallazgo importante, no un problema tuyo: significa que tu test no comprueba lo que crees. Miralo otra vez — probablemente no llama al método que cambiaste.

2 · Falla más de un archivo.

Es normal: el del paso 2 también usa el IVA. Los dos te están avisando de lo mismo.

3.3 Paso 3 · Aislar con un doble

Qué vamos a hacer

Probar el servicio **sin el repositorio real**, poniéndole uno de mentira que devuelve lo que tú digas.

Para entenderlo mejor

El proveedor figurante. Quieres comprobar que el jefe de compras **suma bien**, y para eso necesitas saber exactamente qué hay en el almacén. Si dependes del almacén real, el día que alguien añada un producto tu inspección deja de cuadrar.

El problema

El test del paso 1 usa el repositorio de verdad, y por eso arrastra los números que ese repositorio trae dentro: **5938, 16033, 47504**. Son correctos, y nadie los puede verificar de cabeza.

Con un doble se **elige el precio**: se dice «este producto vale 1000» y la cuenta sale en la pizarra — 1190 con IVA, por tres, menos 10 %, **3213**.

Y ahí está la regla que hay que llevarse, que es la mitad del paso:

Un doble **no** se usa «porque hay una dependencia». Se usa cuando la dependencia real es **lenta, frágil, o no se puede controlar**.

Aquí el motivo es el tercero. En el paso 1 no se daba ninguno de los tres, y por eso allí **no hay doble** — la comparación entre los dos archivos es el contenido.

La alternativa, y por qué no

- **La implementación real**, como en el paso 1: es la que de verdad prueba que las piezas encajan, y no deja elegir los datos.
- **Un doble escrito a mano** (una clase de test que implementa la interfaz): funciona igual de bien, y son treinta líneas más que mantener.
- **Mockito con @Mock**, que es lo de aquí: el doble se declara en dos líneas y **los datos los pone el test**, así que el número que afirmas es el número que elegiste.

Y la extensión: `@ExtendWith(MockitoExtension.class)` y **no** `@SpringBootTest`. Aquí no hay nada que Spring aporte —se prueba una clase con un constructor—, y el contexto de Spring cuesta lo que vas a ver en el paso 5. De regalo, la extensión avisa si preparas una respuesta que el código nunca llega a usar, que es la forma más común de que un test verde no esté probando nada.

Se pega

Archivo **nuevo** practica/src/test/java/cl/dgt/testing/ProductoServiceConDobleTest.java:

```
@Test
void elDescuentoSeCalculaSobreLoQueDevuelveElRepositorio() {
    when(repositorio.porId(1L)).thenReturn(Optional.of(new Producto(1L, "Inventado",
        ↪ 1000)));

    ProductoService servicio = new ProductoService(repositorio);

    assertEquals(3213, servicio.totalConDescuento(1L, 3));
}
```

Fíjate en que el test elige el dato. El 3213 que afirma sale del producto de 1000 pesos que él mismo puso, no de lo que hubiera en el repositorio.

Y fíjate en lo que **no** tiene: ni un `verify(...)`. Lo que importa es el número que sale, no si el método llamó al repositorio una vez o dos. `verify` ata el test a **cómo** está escrito el método, y el día que alguien lo reorganice sin cambiar su comportamiento, el test se pone rojo sin que nada esté mal. Tiene su sitio —un correo que se envía, un pago que se registra: efectos que no devuelven nada— y no es éste.

`new ProductoService(repositorio)` se construye **a mano**, con `new`. Eso sólo se puede hacer porque la dependencia entra por el **constructor**: es el Lab 02 cobrando.

Lo que vas a ver

```
[INFO] Running cl.dgt.testing.ProductoServiceConDobleTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Time elapsed: 0.073 s
```

Vas bien si...

El test pasa en menos de una décima, y los productos con los que trabaja están escritos en el propio test.

Si te atascas

1 · `UnnecessaryStubbingException`

Preparaste un `when(...)` que el código no llega a llamar. **Es un aviso útil**: o el test no prueba lo que crees, o sobra esa preparación.

2 · `NullPointerException` dentro del servicio.

El doble devuelve `null` por defecto para todo lo que no hayas preparado. Te falta un `when(...)`.

3 · `@Mock` no crea nada y todo es `null`.

Falta `@ExtendWith(MockitoExtension.class)` sobre la clase.

3.4 Paso 4 · Los cuatro niveles, y lo que cuesta cada uno

Qué vamos a hacer

Mirar los cuatro tipos de test corriendo juntos y **comparar los tiempos**.

Para entenderlo mejor

La inspección de la mesa suelta tarda un segundo. Abrir la oficina entera con todas las salas tarda mucho más — y a veces es lo único que responde a la pregunta.

El problema

«Probar más» no es gratis. Una suite que tarda diez minutos **no se corre**, y una suite que no se corre no protege de nada. Por eso hay que elegir el nivel.

La alternativa, y por qué no

Nivel	Qué prueba	Qué no ve
Sin Spring	la lógica de una clase	nada del cableado ni de HTTP
Con doble Capa web (@WebMvcTest)	la lógica, con datos elegidos ruta, estado, JSON, manejo de errores	lo mismo no pasa por un servidor real: ni HTTPS, ni cabeceras del contenedor
Todo (@SpringBootTest)	que la aplicación arranca	tarda un orden de magnitud más

El de la capa web es el que más gente se salta, y es el que prueba lo que **sólo** Spring MVC hace: un test unitario del controlador llama a un método y recibe un objeto — **el 404 no aparece por ninguna parte**.

Se corre

```
./mvnw test
```

Lo que vas a ver

```
[INFO] Running cl.dgt.testing.ProductoServiceTest
[INFO] Tests run: 4 ... Time elapsed: 0.089 s

[INFO] Running cl.dgt.testing.ContextoDeSpringTest
[INFO] Tests run: 1 ... Time elapsed: 1.261 s

[INFO] Running cl.dgt.testing.ProductoServiceConDobleTest
[INFO] Tests run: 1 ... Time elapsed: 0.073 s

[INFO] Running cl.dgt.testing.ProductoControllerTest
[INFO] Tests run: 1 ... Time elapsed: 0.359 s

[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
```

(Siete ejecuciones y cuatro métodos de test: el parametrizado cuenta sus cuatro casos.)

Míralo bien: cuatro casos sin Spring tardan 0,089 s. UNO con el contexto entero tarda 1,261 s. Catorce veces más, por un solo test.

Y aun así el caro hace falta, porque es el único que responde a «¿arranca esto?». Si falta un bean, si dos se pelean por el mismo nombre, si una propiedad no resuelve — ninguno de los rápidos se entera.

Los tiempos van a ser distintos en tu máquina. Lo que no cambia es la proporción: el contexto de Spring cuesta un orden de magnitud más que no levantarlo.

Vas bien si...

Tests run: 7, Failures: 0 y puedes señalar en la salida cuál de los cuatro archivos tarda mucho más que los otros tres juntos.

Si te atascas

1 · El test del contexto falla con Failed to load ApplicationContext.

Es exactamente para lo que está. Lee la causa: falta un bean, o hay dos candidatos —el problema del Lab 02—, o una propiedad no resuelve.

2 · La suite entera tarda muchísimo.

Spring reutiliza el contexto entre clases de test que compartan configuración, así que el coste se paga una vez. Pero basta un `@MockitoBean` distinto para que sea otro contexto y otro pago.

4 Lo que aprendiste

1 · Un test que nunca se ha puesto rojo no demuestra nada.

Romper la producción a propósito es la única forma de saber que el test mira donde crees. Cinco minutos, y es lo que separa una red de seguridad de un adorno.

2 · Probar una aplicación Spring no exige levantar Spring.

La mayoría de lo que escribes son clases normales con un constructor. Se prueban como cualquier clase Java, y tardan milésimas.

3 · Se prueban reglas y bordes, no cualquier cosa.

Que el catálogo tenga cuatro productos no merece un test: no es una regla, es el contenido de una lista, y el día que alguien añada uno el test se pone rojo sin que nada esté mal. Lo que merece un test es el descuento por volumen, y los casos que valen son los **bordes**.

4 · Un doble sirve cuando la dependencia real es lenta, frágil o no se puede controlar.

No «siempre que haya una dependencia». Aquí sirvió para elegir el precio y poder comprobar el número de cabeza. Y `verify` no se usó: ata el test a cómo está escrito el método, no a lo que hace.

5 · Cada nivel cuesta, y hay que elegirlo.

0,089 s los cuatro casos sin Spring; 1,261 s uno solo con el contexto entero. Se prueba al nivel más barato que responda la pregunta — y se paga el caro sólo donde hace falta.

5 Para profundizar

- **Invierte los dos `if`** de `totalConDescuento` —pregunta por `>= 3` antes que por `>= 10`— y corre la suite. ¿Cuál de los cuatro casos se pone rojo, y por qué sólo ése?
- **Añade un caso de 5 unidades** al `@CsvSource`. ¿Qué error habría cazado que no cacen los otros cuatro? (Pista: ninguno. Los bordes son donde viven los errores.)
- **Quita `<trimStackTrace>`** del `pom.xml`, rompe el IVA y cuenta las líneas de la traza.
- **Rompe el controlador** —cambia la ruta— y mira cuál de los cuatro archivos de test se entera.
- **Quita el `@ExtendWith`** del test con doble y mira qué error da.

6 Antes de cerrar

Este lab **no deja nada corriendo**: no hay servidor ni base que apagar.

```
./mvnw clean
```

Lo que te llevas:

Un test se escribe, se ve fallar a propósito, y se corre solo. Se prueban reglas y bordes, no cualquier cosa. Un doble sirve cuando la dependencia real estorba, no siempre. Y se prueba al nivel más barato que responda la pregunta.

Lo que queda pendiente, y abre el Lab 09: todos los endpoints que llevas escritos **los puede llamar cualquiera**. En el Lab 09 se cierra la puerta, y se aprende la diferencia entre «no te conozco» y «te conozco, pero esto no es para ti».